Unit (модульне) тестування

в C#

Архітектура тестів · AAA шаблон · Assert · TDD · Практики

#CSharp #NUnit #MSTest #TDD

Зміст лекції

01 **Що таке Unit тест**

Визначення, мета і місце в розробці ПЗ

02 **Типи тестування**

Unit → Integration → System: різниця і коли що використовувати

03 **Верифікація та валідація**

Два фундаментальних поняття контролю якості

04 **Практики та іменування**

Найкращі практики, структура проектів, naming convention

05 **Шаблон AAA + Assert**

Arrange-Act-Assert, методи класу Assert в C#

06 **TDD**

Test-Driven Development: червоний → зелений → рефакторинг

Що таке Unit тест?

Unit тест — блок коду (зазвичай метод), який викликає блок коду, що тестується, і перевіряє правильність його роботи. Якщо результат не співпадає з очікуваним — тест вважається непройденим (failure).

Простий приклад Unit тесту в C# (MSTest):

```
[TestClass]
public class CalculatorTests
{
  [TestMethod]
  public void Add_10Plus20_Returns30()
  {
    // Arrange
    var calc = new Calculator();

    // Act
    int result = calc.Add(10, 20);

    // Assert
    Assert.AreEqual(30, result); // ✅
  }
}
```

Ознаки хорошого Unit тесту:



Швидкий

Виконується за мілісекунди, без зовнішніх ресурсів



Повторюваний

Однаковий результат при будь-якій кількості запусків



Ізольований

Не залежить від БД, мережі чи інших тестів



Читабельний

Назва тесту пояснює що тестується і що очікується

Типи тестування — піраміда тестів

Unit Testing

⚡ Дуже швидко

Тестування кожної атомарної функції окремо, з об'єктами штучно змодельованого середовища (mocks/stubs).

- ✅ Швидко виконуються
- ✅ Легко знайти причину збою
- ✅ Прості у написанні

Integration Testing

🚶 Помірно

Перевірка взаємодії між модулями, підсистемами та зовнішніми компонентами (БД, API).

- ✅ Перевіряє реальну взаємодію
- ✅ Знаходить помилки на межах систем

System Testing

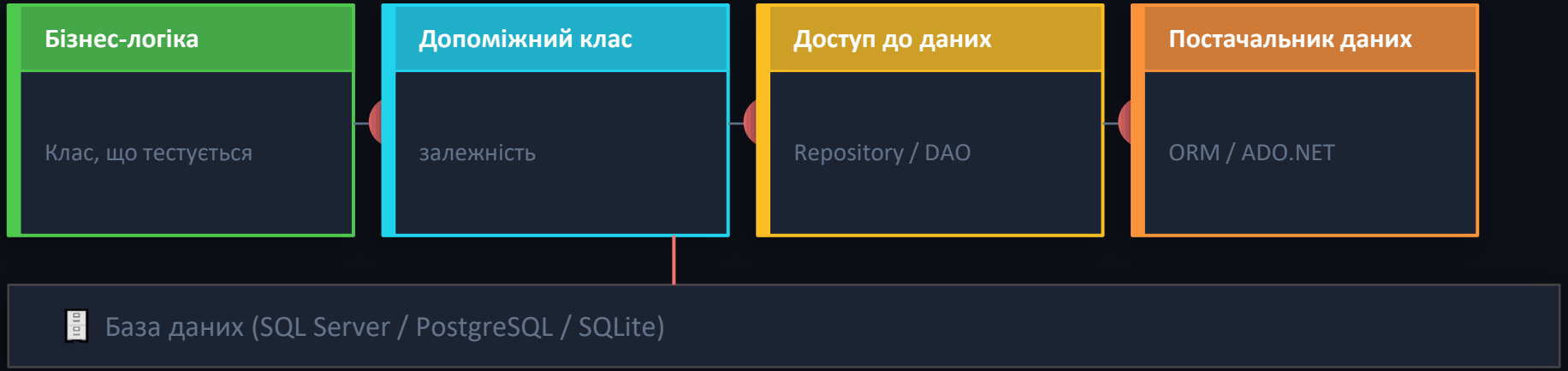
🐢 Повільно

Тестування повної зібраної системи на відповідність функціональним і нефункціональним вимогам.

- ✅ Реальний сценарій використання
- ✅ End-to-end перевірка

Чому інтеграційне тестування складніше?

При інтеграційному тестуванні існує багато точок збою — місць, де поведінка системи не відповідає очікуваній. Це ускладнює пошук справжньої причини помилки.



! = Точка збою: тут система може поводитись несподівано

```
// ✅ Unit тест — один клас, без залежностей:
var repo    = new Mock<IUserRepository>();
var service = new UserService(repo.Object);
repo.Setup(r => r.GetById(1)).Returns(fakeUser);
var result = service.GetUser(1);
Assert.AreEqual("Олена", result.Name); // ✅ ізольовано
```

Верифікація та Валідація

✔ Verification

Верифікація

- Чи робимо ми продукт правильно?
- Чи реалізована вся функціональність?
- Проводиться розробниками
- Інспектування коду, перевірка вимог
- Об'єктивна оцінка реалізованих функцій

🎯 Validation

Валідація

- Чи робимо ми правильний продукт?
- Чи правильно реалізована функціональність?
- Проводиться тестувальниками
- Виконання програми, acceptance tests
- Суб'єктивна оцінка від користувача

💡 *Verification = тести юніт-рівня | Validation = acceptance / системне тестування*

Коли писати Unit тести?

✅ Писати ОБОВ'ЯЗКОВО

- Складна бізнес-логіка без зовнішніх залежностей
- Складні алгоритми (сортування, обчислення, парсинг)
- Код, що зв'язує компоненти між собою
- Критичні шляхи (авторизація, платежі, розрахунки)

⚠ Писати ОБЕРЕЖНО

- Не дуже складний код із залежностями — використовуй mocks
- Legacy-код без тестів — починай з рефакторингу
- Код з частою зміною сигнатури — приймальні тести кращі
- Код, що зав'язаний на UI/форматування

🚫 НЕ ПОТРІБНО

- Тривіальний код без логіки (getters/setters)
- Код з величезною кількістю залежностей без рефакторингу
- Сторонні бібліотеки (вони вже мають свої тести)
- Методи, сигнатура яких постійно змінюється

🔁 ЗОЛОТЕ ПРАВИЛО

- Пишемо тест РАЗОМ з кодом, не після нього
- Не знаєш як точно має працювати — не починай писати
- Тести — частина системи контролю версій (git)
- Автоматизовано: кожен в команді може запустити

Структура проектів та іменування

Структура рішення (Solution)

```
MyApp.sln
├── MyApp.Core/           # бізнес-логіка
├── MyApp.BI/             # business intelligence
├── MyApp.Web/            # UI шар
├── MyApp.Core.Tests/     # ✅ тести для Core
├── MyApp.BI.Tests/       # ✅ тести для BI
└── MyApp.Web.Tests/      # ✅ тести для Web
```

Іменування класів і методів

Клас → тестовий клас:

```
UserManager   →  UserManagerTests
OrderService  →  OrderServiceTests
```

Шаблон іменування методів:

```
[Метод]_[Сценарій]_[ОчікуванийРезультат]
```

Приклади іменування методів тестів:

```
// ✅ Добре — зрозуміло що тестується, за яких умов, що очікується:
Add_10Plus20_Returns30()
GetPasswordStrength_AllChars_Returns5Points()
Divide_ByZero_ThrowsDivideByZeroException()
Login_WrongPassword_ReturnsUnauthorized()

// ❌ Погано — незрозуміло що і чому:
Test1()  /  TestAdd()  /  MyTest()  /  CheckResult()
```

Unit Test Frameworks для C#

NUnit

nunit.org

Найстаріший і найпопулярніший фреймворк. Порт JUnit для .NET. Атрибути: [Test], [TestCase], [SetUp].

```
[TestFixture]

public class MathTests {

  [Test]

  public void Add_Returns_Sum() {

    Assert.That(2+2,
Is.EqualTo(4));
  }

}
```

MSTest

bit.ly/mstest

Офіційний від Microsoft. Вбудований у Visual Studio. Атрибути: [TestClass], [TestMethod], [TestInitialize].

```
[TestClass]

public class MathTests {

  [TestMethod]

  public void Add_Returns_Sum() {

    Assert.AreEqual(4, 2+2);

  }

}
```

xUnit

github.com/xunit

Сучасний мінімалістичний фреймворк. Підтримується .NET Foundation. Атрибути: [Fact], [Theory], [InlineData].

```
public class MathTests {

  [Fact]

  public void Add_Returns_Sum() {

    Assert.Equal(4, 2+2);

  }

}
```

Шаблон AAA — Arrange · Act · Assert

A Arrange

Підготовка: ініціалізація об'єктів, налаштування залежностей, визначення вхідних даних

A Act

Дія: виклик методу, що тестується, з підготовленими параметрами

A Assert

Перевірка: порівняння фактичного результату з очікуваним

Повний приклад AAA з MSTest:

```
[TestClass]
public class BankAccountTests
{
  [TestMethod]
  public void Deposit_100_BalanceIncreases()
  {
    // ——— ARRANGE ———————————————————————
    var account = new BankAccount(initialBalance: 500);
    decimal depositAmount = 100m;
    decimal expected = 600m;

    // ——— ACT ———————————————————————————
    account.Deposit(depositAmount);

    // ——— ASSERT ————————————————————————
    Assert.AreEqual(expected, account.Balance);
```

Те саме в NUnit та xUnit:

```
// NUnit:
[Test]
public void Deposit_100_BalanceOk()
{
  // Arrange
  var acc = new BankAccount(500);
  // Act
  acc.Deposit(100);
  // Assert
  Assert.That(acc.Balance,
    Is.EqualTo(600m)); // ✅
}

// xUnit:
[Fact]
public void Deposit_100_BalanceOk()
```

Атрибути MSTest — lifecycle тестів

[TestClass]

Маркує клас як контейнер тестів

[TestMethod]

Маркує метод як окремий Unit тест

[TestInitialize]

Викликається перед КОЖНИМ тестом (setup)

[TestCleanup]

Викликається після КОЖНОГО тесту (teardown)

[ClassInitialize]

Один раз перед усіма тестами класу

[ClassCleanup]

Один раз після всіх тестів класу

[AssemblyInitialize]

Один раз перед усіма тестами збірки

[AssemblyCleanup]

Один раз після всіх тестів збірки

```
[TestClass] public class MyTests {
```

Assert — клас перевірок у C#

Assert

Порівняння двох значень. Багато перевантажень.

CollectionAssert

Порівняння колекцій та перевірка їх елементів.

StringAssert

Спеціалізовані перевірки для рядків.

Основні методи Assert з прикладами:

```
// AreEqual — рівність значень:
Assert.AreEqual(30, calc.Add(10,20));          // ✅ 30 == 30
Assert.AreEqual("Alice", user.Name);           // ✅ рядки рівні

// AreSame — та сама посилання (reference equality):
Assert.AreSame(expected, actual);               // ✅ той самий об'єкт у пам'яті

// IsTrue / IsFalse:
Assert.IsTrue(user.IsActive);                   // ✅ умова істинна
Assert.IsFalse(cart.IsEmpty());                 // ✅ умова хибна

// InstanceOfType — тип об'єкта:
Assert.IsInstanceOfType(obj, typeof(AdminUser)); // ✅ obj є AdminUser

// CollectionAssert — перевірка колекцій:
CollectionAssert.AreEqual(expected, actual);    // ✅ однакові елементи
CollectionAssert.Contains(list, item);          // ✅ елемент присутній
```

Assert: винятки та параметризовані тести

Тестування виключень (Exceptions):

```
// MSTest — очікуємо виняток:
[TestMethod]
[ExpectedException(typeof(DivideByZeroException))]
public void Divide_ByZero_Throws()
{
    calc.Divide(10, 0); // має кинути виняток
}

// або через Assert.ThrowsException:
Assert.ThrowsException<DivideByZeroException>(
    () => calc.Divide(10, 0)); // ✅ чистіший підхід
```

💡 Коли тест очікує виняток — не використовуй try/catch вручну! Це маскує помилки. Завжди використовуй Assert.ThrowsException<T> або [ExpectedException].

Параметризовані тести ([DataRow] / [Theory]):

```
// MSTest — DataRow:
[DataTestMethod]
[DataRow(10, 20, 30)]
[DataRow(-5, 5,  0)]
[DataRow(0,  0,  0)]
public void Add_Returns(int a, int b, int exp)
{
    Assert.AreEqual(exp, calc.Add(a, b));
}

// xUnit — Theory + InlineData:
[Theory]
[InlineData(10, 20, 30)]
[InlineData(-5, 5,  0)]
public void Add_Works(int a,int b,int exp)
  => Assert.Equal(exp, calc.Add(a,b));
```

TDD — Test-Driven Development

TDD (Test-Driven Development) — підхід розробки ПЗ, при якому Unit тест пишеться ДО коду, що він тестує. Мета: спроектувати API і поведінку ще до реалізації.

🔴 RED

Напиши тест

Пишемо тест для функції, яка ще не існує. Тест падає (fails) — і це нормально.

🟢 GREEN

Напиши код

Пишемо мінімальний код, щоб тест пройшов. Без золотого покриття — просто щоб green.

🔵 REFACTOR

Рефакторинг

Поліпшуємо код не змінюючи поведінку. Тести захищають від регресій.

Демо TDD: Клас Calculator.Add()

```
// 🔴 КРОК 1 — пишемо тест (Calculator ще не існує!):
[TestMethod]
public void Add_10Plus20_Returns30() {
    var calc = new Calculator(); // ❌ compile error!
    Assert.AreEqual(30, calc.Add(10, 20));
}

// 🟢 КРОК 2 — мінімальна реалізація:
public class Calculator {
    public int Add(int a, int b) => a + b; // ✅
```

```
// 🔵 КРОК 3 — рефакторинг (додаємо edge cases):
[DataRow(10, 20,  30)]  // стандартний випадок
[DataRow(-5, -5, -10)]  // від'ємні числа
[DataRow( 0,  0,   0)]  // нулі
[DataTestMethod]
public void Add_Parametrized(int a, int b, int exp)
    => Assert.AreEqual(exp, new Calculator().Add(a,b));

// Всі 3 тести ✅ — можна рефакторити впевнено!
```

Традиційний підхід vs TDD

😐 Традиційний

1. Пишемо метод / клас
2. Пишемо юніт-тест
3. Виконуємо тест
4. Виправляємо помилки

❌ Тест пишеться після — API може бути незручним
❌ Менша мотивація покривати edge cases

🚀 TDD

1. Пишемо тест (🔴 він падає)
2. Пишемо мінімальний код (🟢 green)
3. Рефакторинг (🔵 тести захищають)
4. Повторюємо для кожної фічі

✅ Проектуємо API до реалізації
✅ 100% покриття з самого початку

Навіщо писати Unit тести?



Захист від регресій

Будь-яка зміна коду автоматично перевіряється. Тести знаходять поломки до production.



Краща архітектура

Код, який легко тестувати — зазвичай добре спроектований. Слабка зв'язність = гарна архітектура.



Жива документація

Тести показують як використовувати API. Читабельніші за коментарі у коді.



Впевнений рефакторинг

Можна покращувати код без страху. Тести скажуть якщо щось зламалось.



Рання локалізація

Знайти причину проблеми в ізольованому тесті значно простіше, ніж у production.



Командна робота

Новий розробник може легко зрозуміти що роблять модулі, запустивши тести.

Підсумки лекції



Unit тест

Ізольована перевірка одного метода. Швидко, повторювано, автоматизовано



Піраміда тестів

Unit → Integration → System. 70% тестів мають бути Unit



Verification vs Validation

Verification = правильно пишемо. Validation = пишемо правильне

AAA

Arrange–Act–Assert

Єдиний правильний шаблон структури тесту в C#



Assert

AreEqual, IsTrue, ThrowsException, CollectionAssert, StringAssert



TDD

🔴 Red → 🟢 Green → 🔵 Refactor. Тест ЗАВЖДИ перед кодом

Пишіть тести — пишіть впевнено! 🚀